CHAPTER 06 차원축소

6.1 차원 축소 개요

정의

- 많은 피처로 구성된 다차원 데이터 세트의 차원을 축소하여 새로운 차원의 데이터 세트를 생성하는 것

차원 축소가 필요한 이유

문제	설명
데이터의 희소화	차원이 증가할수록 데이터 포인트 간 거리가 기하급수적으로 멀어져 희소(sparse)한 구조가 됨 → 예측 신뢰도 저하 (차원의 저주)
다중 공선성	피처가 많으면 피처 간 상관관계가 높아질 가능성이 큼 → 선형 회귀 등 선형 모델의 예측 성능 저하
시각화	3차원 이하로 축소하면 데이터를 시각적으로 표현하여 직관적으로 해석 가능
학습 효율	학습 데이터 크기가 줄어 학습에 필요한 처리 능력 감소

피처 선택 vs 피처 추출

구분	피처 선택 (Feature Selection)	피처 추출 (Feature Extraction)
방식	불필요한 피처를 제거하고 데이터 특징을 잘 나타내는 주요 피처만 선택	기존 피처를 저차원의 중요 피처로 압축하여 추출
결과 값	기존 피처 값 그대로	기존 피처와 완전히 다른 값
해당 기법	–	PCA, LDA, SVD, NMF

- 피처 추출은 단순 압축이 아니라 기존 피처가 인지하기 어려운 **잠재적 요소(Latent Factor)** 를 추출하는 것
  - 예) 모의고사 성적, 내신, 수능, 봉사활동, 대외활동 등 → `학업 성취도`, `커뮤니케이션 능력`, `문제 해결력` 등의 잠재 요소로 압축
- 차원 축소의 핵심 의미: 데이터를 더 잘 설명할 수 있는 잠재적 요소를 추출하는 것

활용 분야

- **이미지 데이터**: 많은 픽셀로 구성된 이미지에서 잠재된 특성을 피처로 도출 → 적은 차원으로 학습하여 과적합(overfitting) 감소, 분류 성능 향상
- **텍스트 데이터**: 문서 내 단어들의 구성에서 숨겨진 시맨틱(Semantic) 의미나 토픽(Topic)을 잠재 요소로 추출 → SVD, NMF가 시맨틱 토픽 모델링의 기반 알고리즘

6.2 PCA (Principal Component Analysis)

개요

- 가장 대표적인 차원 축소 기법
- 여러 변수 간 존재하는 상관관계를 이용해 이를 대표하는 **주성분(Principal Component)** 을 추출하여 차원 축소
- 기존 데이터의 정보 유실을 최소화하기 위해 **가장 높은 분산을 가지는 데이터의 축**을 찾아 그 축으로 차원 축소
  - 분산이 데이터의 특성을 가장 잘 나타내는 것으로 간주

축 생성 방식

1. 데이터 변동성(분산)이 가장 큰 방향으로 첫 번째 축 생성
2. 첫 번째 축에 **직교**하는 벡터를 두 번째 축으로 생성
3. 두 번째 축과 직교하는 벡터를 세 번째 축으로 생성 (반복)
4. 생성된 축(벡터)에 원본 데이터를 **투영**하면, 축의 개수만큼의 차원으로 축소됨

→ 원본 피처 개수에 비해 매우 작은 주성분으로 원본 데이터의 총 변동성을 대부분 설명

선형대수 관점

- PCA = 입력 데이터의 **공분산 행렬(Covariance Matrix)을 고유값 분해**하고, 구한 고유벡터에 입력 데이터를 선형 변환하는 것
  - **고유벡터(eigenvector)**: PCA의 주성분 벡터 = 입력 데이터의 분산이 큰 방향
  - **고유값(eigenvalue)**: 고유벡터의 크기 = 입력 데이터의 분산
- **고유벡터**: 행렬 A를 곱해도 방향이 변하지 않고 크기만 변하는 벡터
  - `Ax = ax` (A: 행렬, x: 고유벡터, a: 스칼라 값)
- **공분산 행렬**
  - 분산: 한 변수의 데이터 변동 / 공분산: 두 변수 간의 변동
  - 대각 원소 = 각 변수의 분산, 비대각 원소 = 변수 쌍의 공분산
  - Cov(X, Y) = Cov(Y, X) 이므로 **정방행렬이자 대칭행렬**
- 대칭행렬은 항상 고유벡터를 **직교행렬**로, 고유값을 **정방 대각 행렬**로 대각화 가능

```
C = P Σ Pᵀ

P  : n×n 직교행렬 (열 = 고유벡터 e₁ … eₙ)
Σ  : n×n 정방 대각 행렬 (대각 = 고유값 λ₁ … λₙ)
Pᵀ : P의 전치 행렬
```

- 고유값 λ_1 이 가장 크고 λ_n 이 가장 작음 → e_1 이 가장 분산이 큰 방향, e_2 는 e_1 에 직교하면서 그다음으로 분산이 큰 방향

PCA 수행 단계

1. 입력 데이터 세트의 공분산 행렬 생성

2. 공분산 행렬의 고유벡터와 고유값 계산
3. 고유값이 가장 큰 순으로 K개(PCA 변환 차수)만큼 고유벡터 추출
4. 추출된 고유벡터를 이용해 새롭게 입력 데이터 변환

붓꽃(Iris) 데이터 세트 적용

- 피처 4개(sepal length, sepal width, petal length, petal width) → 2개 주성분으로 축소
- PCA는 여러 속성 값을 연산하므로 속성의 스케일에 영향을 받음 → `StandardScaler`**로 평균 0, 분산 1 변환 후 적용**
- `PCA(n_components=2)` 로 변환 → (150, 2)
- 결과 해석
  - 원본 sepal 피처 2개로 시각화: Setosa는 구분되나 Versicolor, Virginica는 혼재
  - PCA 변환 후: `pca_component_1` 축 기반으로 Setosa 명확히 구분, Versicolor와 Virginica도 대부분 구분
- `explained_variance_ratio_`: 전체 변동성에서 개별 PCA 컴포넌트별로 차지하는 변동성 비율
  - 결과: [0.7296, 0.2285] → 첫 번째 컴포넌트 약 73%, 두 번째 약 23% → 2개 요소로 원본 데이터 변동성의 약 **95%** 설명
- 랜덤 포레스트 분류 정확도 비교 (`cross_val_score`, cv=3)

데이터	피처 수	평균 정확도
원본	4	0.96
PCA 변환	2	0.88

- 속성 개수가 50% 감소한 것을 고려하면 PCA 변환 후에도 원본 데이터의 특성을 상당 부분 유지

신용카드 고객 데이터 세트 (UCI) 적용

- 30,000개 레코드, 피처 23개, Target = `default` (다음 달 연체 여부: 연체 1, 정상 0)
- 전처리: `PAY_0` → `PAY_1`, `default payment next month` → `default` 로 컬럼명 변경
- 상관도 히트맵 (`DataFrame.corr()` + `seaborn.heatmap`)
  - `BILL_AMT1` ~ `BILL_AMT6` 6개 속성 간 상관도 대부분 **0.9 이상**
  - `PAY_1` ~ `PAY_6` 간 상관도도 높음
  - 높은 상관도를 가진 속성들은 소수의 PCA만으로도 자연스럽게 변동성 수용 가능
- `BILL_AMT1~6` 6개 속성 → 2개 컴포넌트로 PCA 변환
  - 변동성 비율: [0.9056, 0.0510]
  - 2개 컴포넌트로 6개 속성 변동성의 약 95% 이상 설명, 첫 번째 축이 약 90% 수용
- 전체 23개 속성 → 6개 컴포넌트로 PCA 변환 후 분류 예측 (랜덤 포레스트, n_estimators=300, cv=3)

데이터	피처 수	평균 정확도
원본	23	0.8170
PCA 변환	6	0.7973

- 전체 속성의 약 1/4 수준 컴포넌트로 예측 성능 1~2% 저하 → PCA의 뛰어난 압축 능력

활용

- 컴퓨터 비전(Computer Vision) 분야에서 활발히 사용
- 얼굴 인식의 **Eigen-face**: PCA 변환으로 원본 얼굴 이미지를 변환하여 사용

6.3 LDA (Linear Discriminant Analysis)

개요

- 선형 판별 분석법. PCA와 매우 유사하게 입력 데이터 세트를 저차원 공간에 투영하여 차원 축소
- 차이점: 지도학습의 **분류(Classification)** 에서 사용하기 쉽도록 **개별 클래스를 분별할 수 있는 기준을 최대한 유지**하면서 차원 축소

구분	PCA	LDA
학습 방식	비지도학습	**지도학습** (결정 값 클래스 필요)
축을 찾는 기준	입력 데이터의 변동성(분산)이 가장 큰 축	입력 데이터의 결정 값 클래스를 최대한 분리할 수 있는 축
분해 대상	공분산 행렬	클래스 간 분산 행렬, 클래스 내부 분산 행렬

원리

- 클래스 분리를 최대화하는 축을 찾기 위해 **클래스 간 분산(between-class scatter)** 과 **클래스 내부 분산(within-class scatter)** 의 비율을 최대화
  - 클래스 간 분산은 **최대한 크게**
  - 클래스 내부 분산은 **최대한 작게**

LDA 수행 단계

1. 클래스 내부와 클래스 간 분산 행렬 계산 (입력 데이터의 결정 값 클래스별 개별 피처의 평균 벡터(mean vector) 기반)
2. 클래스 내부 분산 행렬 S_W, 클래스 간 분산 행렬 S_B 를 다음과 같이 고유벡터로 분해

$$S_W^{-1} S_B = [e_1 \dots e_n] \cdot \mathrm{diag}(\lambda_1 \dots \lambda_n) \cdot [e_1 \dots e_n]^T$$

1. 고유값이 가장 큰 순으로 K개(LDA 변환 차수)만큼 고유벡터 추출
2. 추출된 고유벡터를 이용해 새롭게 입력 데이터 변환

붓꽃 데이터 세트 적용

- 피처 4개를 표준 정규 분포로 스케일링 후 2개 컴포넌트로 변환
- 유의점: LDA는 지도학습이므로 `fit()` **호출 시 결정 값(target)을 함께 입력**
- 결과: LDA로 변환된 데이터 시각화 시 PCA 변환 데이터와 좌우 대칭 형태로 많이 닮아 있음

6.4 SVD (Singular Value Decomposition)

개요

- PCA와 유사한 행렬 분해 기법
- PCA(고유값 분해)는 **정방행렬**만 분해 가능 / SVD는 **행과 열의 크기가 다른 행렬**에도 적용 가능
- m × n 크기의 행렬 A 를 다음과 같이 분해 (특이값 분해)

$$A = U \Sigma V^T$$

구성	크기	설명
U	m × m	왼쪽 특이벡터(singular vector). 모든 특이벡터는 서로 직교
Σ	m × n	대각행렬. 대각 위치 값만 0이 아니고 나머지는 0. 대각 값 = 행렬 A의 **특이값**
V^T	n × n	오른쪽 특이벡터. 서로 직교

SVD 유형

유형	분해 결과 크기	특징
Full SVD	U: m×m, Σ: m×n, V^T: n×n	기본 형태
Compact SVD	U: m×p, Σ: p×p, V^T: p×n	Σ의 비대각 부분과 대각 원소 중 특이값이 0인 부분을 제거하고, 대응하는 U, V 원소도 제거 → 원본 완전 복원 가능
Truncated SVD	U: m×t, Σ: t×t, V^T: t×n	Σ의 대각 원소 중 **상위 t개만** 추출하여 분해 → 원본 근사 복원 (차원 축소)

넘파이 SVD 적용 (4×4 랜덤 행렬)

- `numpy.linalg.svd` 적용 시 U, Sigma, Vt 반환 → 크기 (4, 4), (4,), (4, 4)
- `Sigma`는 0이 아닌 대각 값만 **1차원 배열**로 반환 → `[3.423 2.023 0.463 0.079]`
- 원본 복원 시 `np.diag()`로 0을 포함한 대칭행렬로 변환 후 내적 → 원본 행렬과 동일하게 복원됨
- **로우 간 의존성 부여 시 Sigma 변화**
  - 3번째 로우 = 1번째 로우 + 2번째 로우, 4번째 로우 = 1번째 로우로 업데이트
  - 결과 Sigma: `[2.663 0.807 0. 0.]`

- 이전과 차원은 같지만 Sigma 값 중 2개가 0 → 선형 독립인 로우 벡터가 2개 (행렬의 **랭크 (Rank) = 2**)
- Sigma의 0에 대응하는 U, Sigma, Vt 데이터를 제외하고 복원 가능
- U는 앞 2열, Sigma는 앞 2개 값, Vt는 앞 2행만 사용 → (4, 2) · (2, 2) · (2, 4) 내적으로 원본 복원

사이파이 Truncated SVD 적용

- Truncated SVD는 넘파이가 아닌 **사이파이**에서만 지원
  - 일반 SVD: `scipy.linalg.svd` / Truncated SVD(희소 행렬 지원): `scipy.sparse.linalg.svds`
- 상위 일부 특이값만 사용하므로 원본 행렬을 정확히 복원할 수 없음
  - 데이터 정보가 압축되어 분해되었음에도 상당한 수준으로 원본 행렬 근사 가능
  - 원래 차원의 차수에 가깝게 잘라낼수록(Truncate) 원본에 더 가깝게 복원
- 6×6 행렬 예시: 일반 SVD 시 U, Sigma, Vt = (6, 6), (6,), (6, 6) / Truncated SVD(k=4) 시 (6, 4), (4,), (4, 6) → 복원 결과는 원본에 근사

사이킷런 TruncatedSVD 클래스

- 사이파이 svds와 달리 U, Sigma, Vt 행렬을 반환하지 않음
- PCA 클래스와 유사하게 `fit()`, `transform()` 호출로 원본 데이터를 K개 컴포넌트로 차원 축소
- 원본 데이터를 Truncated SVD 방식으로 분해된 **U × Sigma** 행렬에 선형 변환
- 붓꽃 데이터 결과: PCA와 유사하게 변환 후에도 품종별로 클러스터링이 가능할 정도로 고유성 유지
- **스케일링 후 TruncatedSVD vs PCA 비교**
  - 두 변환 결과 값의 차이 평균 ≈ 2.36e-15, 두 클래스의 `components_` 차이 평균 ≈ 2.08e-17
  - 모두 0에 가까운 값 → 데이터 세트가 스케일링으로 데이터 중심이 동일해지면 SVD와 PCA는 **동일한 변환** 수행
  - PCA가 SVD 알고리즘으로 구현되었음을 의미

구분	PCA	SVD
적용 대상	밀집 행렬(Dense Matrix)만 가능	희소 행렬(Sparse Matrix)도 가능

활용

- 컴퓨터 비전 영역의 이미지 압축을 통한 패턴 인식, 신호 처리 분야
- 텍스트 토픽 모델링 기법인 **LSA(Latent Semantic Analysis)** 의 기반 알고리즘

6.5 NMF (Non-Negative Matrix Factorization)

개요

- Truncated SVD와 같이 낮은 랭크를 통한 행렬 근사(**Low-Rank Approximation**) 방식의 변형
- 원본 행렬 내 모든 원소 값이 **모두 양수(0 이상)** 라는 게 보장되면, 두 개의 기반 **양수 행렬로 분해**하는 기법

```
V (4×6)  ≈  W (4×2)  ×  H (2×6)
```

- **행렬 분해(Matrix Factorization)**: 일반적으로 SVD와 같은 행렬 분해 기법을 통칭
  - 행렬 분해 시 W 행렬과 H 행렬은 일반적으로 길고 가는 행렬 W(원본 행렬의 행 크기와 같고 열 크기보다 작은 행렬)와 작고 넓은 행렬 H(원본 행렬의 행 크기보다 작고 열 크기와 같은 행렬)로 분해
  - 분해된 행렬은 **잠재 요소**를 특성으로 가짐

행렬	의미
W	원본 행에 대해서 이 잠재 요소의 값이 얼마나 되는지에 대응
H	이 잠재 요소가 원본 열(즉, 원본 속성)로 어떻게 구성되었는지를 나타냄

붓꽃 데이터 세트 적용

- 사이킷런 `NMF` 클래스로 2개 컴포넌트 변환 → 변환 결과 값도 모두 0 이상

활용

- SVD와 유사하게 이미지 압축을 통한 패턴 인식, 텍스트의 토픽 모델링 기법, 문서 유사도 및 클러스터링
- **추천(Recommendations)** 영역에서 활발히 적용
  - 사용자의 상품(예: 영화) 평가 데이터 세트인 사용자-평가 순위(user-Rating) 데이터 세트를 행렬 분해 기법을 통해 분해
  - 사용자가 평가하지 않은 상품에 대한 잠재적인 요소를 추출하여 평가 순위(Rating) 예측 → 높은 순위로 예측된 상품 추천
  - 잠재 요소(Latent Factoring) 기반의 추천 방식

요약 비교

구분	PCA	LDA	SVD	NMF
학습 방식	비지도	지도	비지도	비지도
분해 대상	공분산 행렬	$S_W^{-1} S_B$	원본 행렬 (m×n)	원본 행렬 (원소 ≥ 0)
분해 형태	$C = P \Sigma P^T$	고유벡터 분해	$A = U \Sigma V^T$	$V \approx W \times H$
축 선정 기준	분산 최대	클래스 간 분산 / 클래스 내부 분산 최대	특이값 상위	양수 제약 하 근사
사이킷런 클래스	`PCA`	`LinearDiscriminantAnalysis`	`TruncatedSVD`	`NMF`

구분	PCA	LDA	SVD	NMF
`fit()` 입력	X	X, y	X	X
주요 활용	컴퓨터 비전 (Eigen-face)	분류	희소 행렬, 텍스트 (LSA)	토픽 모델링, 추천 시스템